ETH
Swiss Federal Institute of Technology Zurich

Department of Mathematics

Semester Thesis Spring 2026

Benjamin Gasparini

Construction, Classification, and Optimization of Generators of $H_1(\Gamma_h, \mathbb{Z})$ for Triangulated Surfaces

Submission Date: 31 May 2026

Advisor: Prof. Dr. R. Hiptmair

Abstract

We implement the algorithm from Hiptmair and Ostrowski (2002) to construct generators of $H_1(\Gamma_h, \mathbb{Z})$ bounding with respect to the exterior. Implementation is done in python where $\mathbb{R}^3$-embedded triangulated surface meshes Γ_h are created using Gmsh. We further extend the algorithm with the goal of constructing generators of shorter length. This is done by considering different spanning tree construction strategies in the algorithm as well as using supplemental post-processing to shorten the generators. Finally a visualization of the surfaces and the constructed generators is provided in Gmsh.

Contents

Chapter 1

Introduction

1.1 Motivation

Hiptmair and Ostrowski (2002) provides an algorithm to construct generators of $H_1(\Gamma_h, \mathbb{Z})$ that are bounding with respect to the exterior (Γ_h is a triangulated surface mesh) which finds application in the field of computational electromagnetism. We intended to implement this algorithm while considering the additional task of decreasing the length of the constructed generators. In this chapter the relevant background and the algorithm from Hiptmair and Ostrowski (2002) are summarized. The succeeding chapters will then tackle different techniques aimed at producing shorter generators. All relevant code is made available at the following link.

1.2 Homology Group and Bounding Cycles

Let $\Omega \subset \mathbb{R}^3$ be a Lipschitz-polyhedron. We consider a triangulated surface mesh $\Gamma_h \subset \mathbb{R}^3$ of $\partial\Omega$. Let $\mathcal{S}_0$ denote the set of vertices, $\mathcal{S}_1$ the set of edges and $\mathcal{S}_2$ the set of faces of Γ_h. We will now define some concepts from algebraic topology as they relate to Γ_h:

Definition 1.1. *For $l \in \{0, 1, 2\}$ we define the surface l-chains as the mappings $\gamma : \mathcal{S}_l \to \mathbb{Z}$. A surface l-chain γ can be written as a sum*

$$\gamma := \sum_{x \in \mathcal{S}_l} \alpha_x x$$

where $\forall x \in \mathcal{S}_l : \alpha_x := \gamma(x) \in \mathbb{Z}$. We denote by $C_l(\Gamma_h, \mathbb{Z})$ the set of surface l-chains.

Remark. It is easy to check that $C_l(\Gamma_h, \mathbb{Z})$ is a free abelian group under addition.

An element $x \in \mathcal{S}_l$ is called an l-simplex. Every l-simplex has an interior orientation given by an ordering of its vertices. For $x \in \mathcal{S}_l$, $y \in \mathcal{S}_k$ we write $x \prec y$ if every vertex of x is also a vertex of y, we then say x is contained in y. We can now define the incidence relation $\iota(x, y)$ as follows:

Definition 1.2. *If x is not contained in y we set $\iota(x, y) := 0$.*

If $x \prec y$ and the interior orientation of x matches/differs from the ordering of its vertices in y we set $\iota(x, y) := \pm 1$.

We are now equipped to define the boundary homomorphism:

Definition 1.3. *For $l \in \{1,2\}$ and an l-simplex x we define its boundary by*

$$\partial_l x := \sum_{y \in \mathcal{S}_{l-1}} \iota(y,x) y \in C_{l-1}(\Gamma_h, \mathbb{Z})$$

Then the boundary homomorphism $\partial_l : C_l(\Gamma_h, \mathbb{Z}) \to C_{l-1}(\Gamma_h, \mathbb{Z})$ is defined by

$$\partial_l(\sum_{x \in \mathcal{S}_l} \alpha_x x) := \sum_{x \in \mathcal{S}_l} \alpha_x \partial_l x$$

Remark. It is easy to check that for $l \in \{1,2\}$ ∂_l is indeed a group homomorphism.

We thus get two subgroups $Z_1(\Gamma_h, \mathbb{Z}) := \mathrm{Ker}(\partial_1)$ and $B_1(\Gamma_h, \mathbb{Z}) := \mathrm{Im}(\partial_2)$ of $C_1(\Gamma_h, \mathbb{Z})$ which we refer to as 1-cycles and 1-boundaries respectively.

It follows from the definition of $\iota(x,y)$ that $B_1(\Gamma_h, \mathbb{Z}) \subseteq Z_1(\Gamma_h, \mathbb{Z})$. Thus we can define the first homology group as follows:

Definition 1.4. *We call the quotient group $H_1(\Gamma_h, \mathbb{Z}) := Z_1(\Gamma_h, \mathbb{Z})/B_1(\Gamma_h, \mathbb{Z})$ the first homology group.*

Two 1-cycles are called homologous if they have the same class in $H_1(\Gamma_h, \mathbb{Z})$.

We denote the exterior by $\Omega' := \mathbb{R}^3 \setminus \Omega$.

Definition 1.5. *A 1-cycle $\gamma \in Z_1(\Gamma_h, \mathbb{Z})$ is called bounding with respect to the exterior Ω' if $\gamma \in B_1(\Omega', \mathbb{Z})$.*

1.3 Construction of Generators

Definition 1.6. *We define an (undirected) graph as a triple $G = (V, E, \phi)$ where V is a set of vertices, E a set of edges and $\phi : E \to \{\{x,y\} | x,y \in V\}$ is called an incidence function.*

The triangulated surface mesh Γ_h can be interpreted as a graph $G = (\mathcal{S}_0, \mathcal{S}_1, \phi)$ on the vertices $\mathcal{S}_0$, where $\forall e \in \mathcal{S}_1 : \phi(e) := \{x,y\}$ s.t. $x \prec e \wedge y \prec e$.

We further introduce the dual graph $D = (\mathcal{S}_2, \mathcal{S}_1, \psi)$ on the faces $\mathcal{S}_2$ of Γ_h, where $\forall e \in \mathcal{S}_1 : \psi(e) := \{x,y\}$ s.t. $e \prec x \wedge e \prec y$.

Remark. Since every edge of Γ_h clearly contains exactly two vertices and is contained in exactly two triangles the incidence functions ϕ and ψ are well-defined.

Let $(\Gamma_h^i)_{i=1}^p$ be the connected components (in a topological sense) of Γ_h. By the definitions of the two graphs $(\Gamma_h^i)_{i=1}^p$ correspond to the connected components $(G_i)_{i=1}^p$ and $(D_i)_{i=1}^p$ (in a graph theoretical sense) of G and D. We write $G_i = (V_i, E_i, \phi|_{E_i})$ and $D_i = (F_i, E_i, \psi|_{E_i})$ (s.t. $\bigcup_{i=1}^p V_i = \mathcal{S}_0$, $\bigcup_{i=1}^p E_i = \mathcal{S}_1$ and $\bigcup_{i=1}^p F_i = \mathcal{S}_2$).

Let $i \in \{1, .., p\}$. Let T_{D_i} be a spanning tree of D_i. Let $E_i' \subseteq E_i$ be the subset of edges not contained in T_{D_i}.

Lemma 1.7. *The subgraph* $G_i' := (V_i, E_i', \phi|_{E_i'})$ *of* G_i *is connected.*

Proof: See Hiptmair and Ostrowski (2002). □

Thus we can find a spanning tree $T_{G_i'}$ of G_i'. Let $E_i^* \subseteq E_i'$ be the subset of edges not contained in $T_{G_i'}$. Let $e \in E_i^*$ arbitrary with $\{x, y\} := \phi|_{E_i^*}(e) \subseteq V_i$. Since $T_{G_i'}$ is a spanning tree we can find a unique path from x to y comprised only of edges in $T_{G_i'}$. Adding the edge e to this path gives us a cycle which we will denote by s_e.

Remark. The algorithm given in Hiptmair and Ostrowski (2002) to construct the set E_i^* does not work. Instead use spanning tree algorithms to construct $T_{G_i'}$, then choose E_i^* as defined above. Furthermore the given algorithm to obtain cycles s_e from edges $e \in E_i^*$ is also incorrect. A working version is provided below:

For an arbitrary root vertex $v \in V_i$ we can define a depth function depth : $V_i \to \mathbb{N}$ s.t. depth(v') is the length of the unique path from v to v' in $T_{G_i'}$. If we use BFS or DFS to construct $T_{G_i'}$ we automatically have access to such a function (see Chapter 2).

Algorithm 1: Create cycles $(s_e)_{e \in E_i^*}$

Input: Tree $T_{G_i'}$, depth function depth : $V_i \to \mathbb{N}$, edge set E_i^*,
Output: cycles $(s_e)_{e \in E_i^*}$

```
for e ∈ E_i^* do
    s_e ← empty list of edges;
    s_e.push_back(e);
    x, y ← vertices of e;
    repeat
        while depth(x) > depth(y) do
            for z adjacent to x in T_{G_i'} do
                if depth(x) > depth(z) then
                    e' ← edge between {x, z};
                    s_e.push_front(e');
                    x ← z;
                    break;
                end
            end
        end
        while depth(x) ≤ depth(y) do
            for z adjacent to y in T_{G_i'} do
                if depth(y) ≤ depth(z) then
                    e' ← edge between {y, z};
                    s_e.push_back(e');
                    y ← z;
                    break;
                end
            end
        end
    until x = y;
end
return (s_e)_{e ∈ E_i^*};
```

We can turn the cycles s_e into 1-cycles $\gamma_e := \sum_{x \in s_e} \alpha_x x$ by setting $\alpha_e := 1$ and then sequentially visiting each x on s_e while defining $\alpha_x := \pm 1$ depending on the interior orientation of x.

Theorem 1.8. *$(\gamma_e)_{e \in E_i^*}$ represents a basis of $H_1(\Gamma_h^i, \mathbb{Z})$.*

Proof: See Hiptmair and Ostrowski (2002). □

Let now $\gamma \in Z_1(\Gamma_h, \mathbb{Z})$ arbitrary. We can write $\gamma = \sum_{i=1}^p \gamma_i$ with $\gamma_i \in C_1(\Gamma_h^i, \mathbb{Z})$. Indeed we have $\gamma_i \in Z_1(\Gamma_h^i, \mathbb{Z})$ (since the fact that no two edges in different connected components can share a vertex implies that $\partial_1(\gamma) = 0 \iff \forall i \in \{1, .., p\} : \partial_1(\gamma_i) = 0$). Similarly if we also assume $\gamma \in B_1(\Gamma_h, \mathbb{Z})$ then $\forall i \in \{1, .., p\} : \gamma_i \in B_1(\Gamma_h^i, \mathbb{Z})$ (since otherwise two faces in different connected components would have to share an edge). Thus we have that

$$H_1(\Gamma_h, \mathbb{Z}) = Z_1(\Gamma_h, \mathbb{Z})/B_1(\Gamma_h, \mathbb{Z}) = \bigoplus_{i=1}^p Z_1(\Gamma_h^i, \mathbb{Z})/B_1(\Gamma_h^i, \mathbb{Z}) = \bigoplus_{i=1}^p H_1(\Gamma_h^i, \mathbb{Z})$$

So we have indeed found a basis $(\gamma_e)_{e \in E^*}$ of $H_1(\Gamma_h^i, \mathbb{Z})$ (where $E^* := \bigcup_{i=1}^p E_i^*$).

Setting $N := |E^*|$ we will hence forth denote this basis by $(\gamma_i)_{i=1,..,N}$.

1.4 Finding Bounding Cycles

Definition 1.9. *We define the linking number $L(\gamma, \gamma')$ of two piecewise differentiable disjoint $\gamma, \gamma' \in Z_1(\mathbb{R}^3, \mathbb{Z})$ as*

$$L(\gamma, \gamma') := -\int_\gamma \int_{\gamma'} \frac{1}{4\pi \|x - y\|} d\vec{s}(x) \times d\vec{s}(y)$$

In order to make use of the linking number for 1-cycles in $Z_1(\Gamma_h, \mathbb{Z})$ we must make sure that they are disjoint. To that end we introduce the following concept:

Definition 1.10. *Let $\gamma \in Z_1(\Gamma_h, \mathbb{Z})$. We define the submerged cycle of γ as its homology class $\gamma_\downarrow \in H_1(\bar{\Omega}, \mathbb{Z})$.*

Given our constructed basis $(\gamma_i)_{i=1,..,N}$ of $H_1(\Gamma_h, \mathbb{Z})$ we can now use the following corollary from Hiptmair and Ostrowski (2002) to identify 1-cycles bounding with respect to the exterior:

Corollary 1. *Let $\gamma \in Z_1(\Gamma_h, \mathbb{Z})$. γ is bounding w.r.t. $\Omega' \iff \forall i \in \{1, .., N\} : L(\gamma_{i\downarrow}, \gamma) = 0$.*

Proof: See Hiptmair and Ostrowski (2002). □

Since $(\gamma_i)_{i=1,..,N}$ is a basis any 1-cycle $\gamma \in H_1(\Gamma_h, \mathbb{Z})$ can be written as a linear combination $\gamma = \sum_{j=1}^N \beta_j \gamma_j$ (for some $\beta_j \in \mathbb{Z}$). By linearity of the integral it follows that

$$L(\gamma_{i\downarrow}, \gamma) = \sum_{j=1}^N \beta_j L(\gamma_{i\downarrow}, \gamma_j) = (K^\top \beta)_i$$

where $K^\top := (L(\gamma_{i\downarrow}, \gamma_j))_{i,j=1,...,N} \in \mathbb{Z}^{N \times N}$ and $\beta := (\beta_1, ..., \beta_N)^\top \in \mathbb{Z}^N$.

Using the above Corollary 1 it follows that any 1-cycle bounding w.r.t. the exterior is induced by an integer vector β with $\beta \in \mathrm{Ker}(K^\top)$. Consequentially an integral basis of $\mathrm{Ker}(K^\top)$ induces a basis for the desired 1-cycles.

To calculate the matrix K we must first construct the submerged cycles $(\gamma_{i\downarrow})_{i=1,\dots,N}$. That can be done using the following algorithm:

Algorithm 2: Create submerged cycles $(\gamma_{i\downarrow})_{i=1,\dots,N}$

Input: edge cycles $(s_i)_{i=1,..,N}$ corresponding to 1-cycles $(\gamma_i)_{i=1,\dots,N}$, normal vectors $(n_x)_{x\in\mathcal{S}_0}$ for each vertex on the surface Γ_h, small $\epsilon > 0$

Output: polygons $(U_i)_{i=1,..,N}$ representing the submerged cycles $(\gamma_{i\downarrow})_{i=1,\dots,N}$

```
for i ∈ {1, .., N} do
    U_i ← empty list of points in ℝ³;
    for vertex x visited by s_i do
        v ← coordinates of x in ℝ³;
        U_i.push_back(v − ϵ ∗ n_x);
    end
end
return (U_i)_{i=1,..,N};
```

Remark. $\epsilon > 0$ has to be chosen sufficiently small s.t. $U_i \subset \Omega$ for all i.

Hiptmair and Ostrowski (2002) provides a different algorithm for submerged cycles using a tetrahedral triangulation of Ω. Since our implementation only has access to vertices and triangles on the surface we use vertex normals instead.

We omit using shifted cycles as described in Hiptmair and Ostrowski (2002) as submerging is sufficient to achieve disjointedness and compute the linking number. Nevertheless and implementation that uses shifted cycles is provided in the code as well.

Chapter 2

Spanning Tree Algorithms

For the construction of a basis $(\gamma_i)_{i=1,..,N}$ of $H_1(\Gamma_h, \mathbb{Z})$ as described in Section 1.3 it is necessary to create spanning forests $(T_{D_i})_{i=1}^p$ and $(T_{G_i'})_{i=1}^p$. In this chapter we will look at different algorithms to construct these forests with the aim of finding a strategy that results in short edge cycles $(s_i)_{i=1,..,N}$. We compare the performance of these algorithms on eight different surface meshes (see the code on Github to observe the meshes and the constructed generators of $H_1(\Gamma_h, \mathbb{Z})$).

2.1 Depth-First Search (DFS)

A well known algorithm for spanning tree construction is Depth-First Search (for example seen in Cormen et al. (2009)). See algorithm 3 below.

Algorithm 3: DFS

Input: Graph $G = (V, E, \phi)$

Output: edge set $E_{\text{Tree}} \subseteq E$ defining a spanning forest of G, depth function $\text{depth} : V \to \mathbb{N}$

Function `DFS`(*vertex* $v \in V$, *vertex set* $\tilde{V} \subseteq V$, *edge set* $E_{Tree} \subseteq E$, *depth function* $depth : V \to \mathbb{N}$)**:**

```
    for vertex v' adjacent to v in G do
        e ← edge between {v, v'};
        if v' ∈ Ṽ then
            E_Tree ← E_Tree ∪ {e};
            Ṽ ← Ṽ \ {v'};
            depth(v') ← depth(v) + 1;
            DFS(v', Ṽ, E_Tree, depth)
        end
    end
E_Tree ← ∅;
Ṽ ← V;
while Ṽ ≠ ∅ do
    v ← some element of Ṽ;
    Ṽ ← Ṽ \ {v};
    depth(v) ← 0;
    DFS(v, Ṽ, E_Tree, depth)
end
return E_Tree, depth;
```

We can use DFS to construct the forests $(T_{D_i})_{i=1}^{p}$ and $(T_{G_i'})_{i=1}^{p}$. It becomes clear from visual observation alone that the thus obtained generators are far from optimal in terms of length. See table 2.1 below for the results:

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	29, 9	54, 39, 13, 9	55, 59, 47, 40, 7, 10	39, 34, 53, 38, 76, 8	19, 43, 41, 40	95, 7, 48, 44, 9, 66, 16, 11, 75, 93, 8, 70, 50, 11	70, 85, 121, 10, 46, 6	14, 12, 67, 67, 42, 17, 44, 14, 71, 54
total length:	38	115	218	248	143	603	338	402

Table 2.1: Edge lengths of the cycles obtained by using DFS

Remark. In addition to the meshes 1 to 8 the code also provides meshes 0 and 9.

2.2 Breadth-First Search (BFS)

The method proposed in Hiptmair and Ostrowski (2002) is the Breadth-First Search algorithm (algorithm 4 below):

Algorithm 4: BFS

Input: Graph $G = (V, E, \phi)$

Output: edge set $E_{\text{Tree}} \subseteq E$ defining a spanning forest of G, depth function $\text{depth} : V \to \mathbb{N}$

```
E_Tree ← ∅;
Ṽ ← V;
Q ← empty queue of vertices;
while Ṽ ≠ ∅ do
    v_root ← some element of Ṽ;
    Ṽ ← Ṽ \ {v_root};
    depth(v_root) ← 0;
    Q.push_back(v_root);
    while Q ≠ ∅ do
        v ← Q.pop_front();
        for vertex v' adjacent to v in G do
            e ← edge between {v, v'};
            if v' ∈ Ṽ then
                E_Tree ← E_Tree ∪ {e};
                Ṽ ← Ṽ \ {v'};
                depth(v') ← depth(v) + 1;
                Q.push_back(v');
            end
        end
    end
end
return E_Tree, depth;
```

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	9, 19	12, 39, 9, 20	30, 30, 7, 29, 7, 5	20, 18, 52, 7, 6, 8	9, 33, 6, 28	10, 38, 40, 41, 43, 42, 6, 32, 8, 6, 34, 6, 6, 43	38, 20, 27, 6, 5, 5	32, 36, 45, 10, 33, 35, 28, 15, 33, 49
total length:	28	80	108	111	76	355	101	316

Table 2.2: Edge lengths of the cycles obtained by using BFS

We notice that using BFS to construct both forests consistently produces significantly shorter cycles (see table 2.2) than using DFS (table 2.1). It is thus a good starting point, but it will be seen that further refinement of the algorithm is still possible.

2.3 Graph Centers

When performing BFS on a graph for each connected components the algorithm starts at an arbitrary root vertex v_{root} (see algorithm 4). Instead of a random pick we will attempt

to find choices for the vertices v_{root} that yield short cycles.

Let $G = (V, E, \phi)$ be a connected graph. For vertices $x, y \in V$ we set the distance $d(x, y) \in \mathbb{N}$ to be the length of a shortest path from x to y in G.

Theorem 2.1. *Let $G = (V, E, \phi)$ be a connected graph and $v_{root} \in V$. Let $depth : V \to \mathbb{N}$ be the depth function obtained by performing BFS on G starting at v_{root}. Then $\forall x \in V$: $depth(x) = d(v_{root}, x)$.*

Proof: See Cormen et al. (2009). □

We remember from section 1.3 that a cycle s_e is constructed from an edge e on vertices $\{x, y\}$ by finding the unique path $p = (v_1, .., v_l)$ (with $x = v_1, y = v_l$) in a spanning tree $T_{G'_i}$ of a connected graph G'_i. Assume that $T_{G'_i}$ was constructed via BFS from a starting vertex v_{root}. Then clearly the path p has length $l \leq \text{depth}(x) + \text{depth}(y) = d(v_{\text{root}}, x) + d(v_{\text{root}}, y) \leq 2 \max_{z \in V_i} d(v_{\text{root}}, z)$. So we can limit the length of the path p (and thus of the cycle s_e) by choosing a $v_{\text{root}} \in V_i$ that minimizes $\max_{z \in V_i} d(v_{\text{root}}, z)$. This is precisely the definition of the graph center (for example as described in Diestel (2017)):

Definition 2.2. *Let $G = (V, E, \phi)$ be a connected graph. A vertex $v \in V$ is called a graph center of G if its farthest distance to another vertex is a short as possible, i.e. if $v := \arg\min_{x \in V} \max_{y \in V} d(x, y)$.*

To obtain a graph center one can use the following algorithm:

Algorithm 5: Find Graph Center

Input: a connected graph $G = (V, E, \phi)$
Output: a graph center v_{central} of G
$x \leftarrow$ an arbitrary vertex in V;
$\text{depth}_x \leftarrow$ depth function obtained by BFS starting from $v_{root} := x$;
$y \leftarrow \arg\max_{y \in V} \text{depth}_x(y)$;
$E_{\text{Tree}}, \text{depth}_y \leftarrow$ BFS starting from $v_{root} := y$;
$z \leftarrow \arg\max_{z \in V} \text{depth}_y(z)$;
$v_1, ..., v_n \leftarrow$ unique path from y to z in E_{Tree} (i.e. $v_1 = x, v_n = y$);
$v_{\text{central}} \leftarrow v_{\lceil \frac{n}{2} \rceil}$;
return $v_{central}$;

So we have come up with the following strategy: Construct the forest $(T_{D_i})_{i=1}^p$ using BFS as in section 2.2. Then find a graph center v_i for each connected graph G'_i (as defined in section 1.3) and construct each tree $T_{G'_i}$ by performing BFS starting from the found central vertex v_i.

See table 2.3 below for the results:

We notice that the results are comparable to what we got by just using BFS without utilizing graph centers. This could be explained by the fact that our available options for the trees $(T_{G'_i})_{i=1}^p$ are already significantly restricted by the trees $(T_{D_i})_{i=1}^p$. This is something we will take into consideration for our next approach.

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	9, 19	12, 39, 9, 20	37, 37, 29, 7, 7, 5	52, 18, 20, 7, 6, 8	9, 6, 28, 27	48, 34, 40, 54, 50, 35, 32, 56, 8, 6, 6, 6, 5, 36	27, 6, 20, 34, 5, 5	45, 32, 12, 28, 28, 10, 15, 11, 32, 21
total length:	28	80	122	111	70	416	97	243

Table 2.3: Edge lengths of the cycles obtained by using BFS for the trees $(T_{D_i})_{i=1}^p$ and BFS from graph centers for the trees $(T_{G_i'})_{i=1}^p$

2.4 Kruskal's Algorithm

Instead of a graph center v_i' of G_i' we can find a "better" graph center v_i of G_i before constructing the tree T_{D_i}. We will then attempt to build the tree T_{D_i} in such a way that v_i is still available as a graph center for G_i' (remember that G_i' is the graph G_i without the edges of T_{D_i}). To that end we introduce Kruskal's algorithm for spanning forests (for example described in Cormen et al. (2009)):

Algorithm 6: Kruskal

Input: Graph $G = (V, E, \phi)$, weight function weight : $E \to \mathbb{Z}$
Output: edge set $E_{\text{Tree}} \subseteq E$ defining a spanning forest of G
$E_{\text{Tree}} \leftarrow \emptyset$;
introduce function set : $V \to \mathcal{P}(V)$;
$\forall v \in V : \text{set}(v) \leftarrow \{v\}$;
$e_1, ..., e_{|E|} \leftarrow$ ordering of E s.t. $\forall i \leq j : \text{weight}(e_i) \leq \text{weight}(e_j)$;
for $i \in \{1, .., |E|\}$ **do**
  $\{x, y\} \leftarrow \phi(e_i)$;
  if $set(x) \neq set(y)$ **then**
    $E_{\text{Tree}} \leftarrow E_{\text{Tree}} \cup \{e_i\}$;
    $\text{set}(x), \text{set}(y) \leftarrow \text{set}(x) \cup \text{set}(y)$;
  end
end
return E_{Tree};

For each edge set E_i we propose the following weight function: Let $v_i \in V_i$ be a graph center of G_i. $\forall e \in E_i$ we define $\text{weight}(e) := -\min_{x \in \phi(e)} d(v_i, x)$. I.e. edges that are farther away from the graph center v_i have smaller weights.

Using Kruskal's algorithm with the above weights to construct the forest $(T_{D_i})_{i=1}^p$ and then BFS from graph centers (as in section 2.3) for the forest $(T_{G_i'})_{i=1}^p$ yields the following cycle lengths (see table 2.4):

We observe that this strategy consistently produces cycles of significantly shorter length than all our previous approaches.

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	5, 16	19, 8, 15, 6	9, 6, 12, 12, 8, 12	11, 17, 17, 21, 7, 6	6, 13, 12, 6	24, 6, 28, 26, 6, 6, 8, 14, 26, 22, 20, 20, 6, 32	9, 16, 8, 9, 15, 18	18, 24, 16, 16, 5, 7, 18, 11, 7, 20
total length:	21	48	59	79	37	244	75	142

Table 2.4: Edge lengths of the cycles obtained by using Kruskal with the described weights for the trees $(T_{D_i})_{i=1}^{p}$ and BFS from graph centers for the trees $(T_{G'_i})_{i=1}^{p}$

Chapter 3

Shortening of Cycles

Let s be an edge cycle on a connected graph G_i that induces a 1-cycle γ.[1] We want to find a way to replace s by a shorter edge cycle $\tilde{s}$ such that the 1-cycle induced by $\tilde{s}$ remains homologous to γ (this problem is for example discussed in Colin de Verdière and Erickson (2010)). We look at the following approach:

Let $p = e_1...e_l$ be a path contained in s from a vertex $x \in V_i$ to a vertex $y \in V_i$. We furthermore assume that p does not visit any vertex more than once, i.e. that it is indeed a path. Let $\tilde{p} = \tilde{e}_1...\tilde{e}_{\tilde{l}}$ be another path from x to y in G_i s.t. $\tilde{l} < l$ and s.t. $\tilde{p}$ is disjoint from p (apart from the vertices x, y). Clearly we can replace p by $\tilde{p}$ in s resulting in a shorter edge cycle $\tilde{s}$. But is the 1-cycle $\tilde{\gamma}$ induced by $\tilde{s}$ homologous to γ?

To answer this question we look at the difference of the two 1-cycles $\tilde{\gamma} - \gamma$ which is represented by the graph cycle $c = \tilde{e}_1...\tilde{e}_{\tilde{l}}e_l...e_1$. It is to be verified whether c represents a 1-boundary. To that end we look at the dual graph $D_i = (F_i, E_i, \psi|_{E_i})$. Let $\tilde{D}_i := (F_i, \tilde{E}_i, \psi|_{\tilde{E}_i})$ with $\tilde{E}_i := E_i \setminus \{\tilde{e}_1, ..., \tilde{e}_{\tilde{l}}, e_1, ..., e_l\}$, i.e. the dual graph with the edges in c removed. Assume $\tilde{D}_i$ has exactly two connected components given by F_i^1, F_i^2 (with $F_i^1 \cup F_i^2 = F_i$). Using the implied direction of c we note that clearly all faces on one side of c lie in the same connected component of $\tilde{D}_i$. Thus we have in particular that c represents the boundary of the 2-chain induced by F_i^1 (since every edge that is not in c is incident to exactly 2 or 0 faces of F_i^1).

So we have seen that the replacement of s by $\tilde{s}$ is allowed if $\tilde{D}_i$ has exactly two connected components. We thus propose to attempt to shorten s according to the following strategy:

[1] s does not have to be a true graph theoretical cycle in the sense that it may visit the same vertices and edges more than once (specifically if γ has edge weights of absolute value larger than 1). Note also that a 1-cycle γ may be induced by more than just one edge cycle s (e.g. if γ has edges with non-zero weight that lie in distinct connected components).

Algorithm 7: shorten an edge cycle s without changing homology class

Input: connected graph $G_i = (V_i, E_i, \phi|_{E_i})$, dual graph $D_i = (F_i, E_i, \psi|_{E_i})$, edge cycle s on E_i that represents a 1-cycle γ

Output: a shorter cycle $\tilde{s}$ on E_i that represents a 1-cycle homologous to γ

$\tilde{s} \leftarrow s$;
$l \leftarrow 2$;
return $\tilde{s}$;
while $l <$ *length of* $\tilde{s}$ **do**
  for *path* $p = e_1, .., e_l$ *in* $\tilde{s}$ **do**
    $x, y \leftarrow$ endpoints of p;
    $\tilde{G}_i \leftarrow G_i$ with all vertices of p except x, y removed;
    $\tilde{p} = \tilde{e}_1, .., \tilde{e}_{\tilde{l}} \leftarrow$ shortest path from x to y in $\tilde{G}_i$ (obtained using BFS);
    if $\tilde{l} < l$ **then**
      $\tilde{D}_i \leftarrow (F_i, \tilde{E}_i, \psi|_{\tilde{E}_i})$ with $\tilde{E}_i := E_i \setminus \{\tilde{e}_1, ..., \tilde{e}_{\tilde{l}}, e_1, ..., e_l\}$;
      count $\leftarrow$ number of components of $\tilde{D}_i$ (obtained by creating a spanning forest of $\tilde{D}_i$ and counting the number of trees);
      if *count* $= 2$ **then**
        $\tilde{s} \leftarrow \tilde{s}$ with p replaced by $\tilde{p}$;
        $l \leftarrow 1$;
        break;
      end
    end
  end
  $l \leftarrow l + 1$;
end

Remark. In practice we can observe that with increasing path length l it also becomes increasingly unlikely to find a viable shorter replacement path. Thus the implementation in the code allows to terminate the algorithm once l reaches a given $l_{\max}$. Furthermore it can be seen that a viable replacement path will usually have length $\tilde{l} = l - 1$ (a significantly shorter path is unlikely as we have looked at the smaller l already). Thus the code also allows to restrict to these cases to save computational cost.

We can for instance apply algorithm 7 to the very long cycles in table 2.1 representing a basis of $H_1(\Gamma_h, \mathbb{Z})$. See table 3.1 below for the result:

We note that the described optimization strategy significantly decreases the length of these cycles, generally cutting it about in half. However they are for the most part still inferior to the ones obtained in section 2.4 without optimization. Further applying algorithm 7 to the cycles in section 2.4 yields the following result (see table 3.2):

Remark. Note that here we have applied optimization to the cycles representing a full basis of $H_1(\Gamma_h, \mathbb{Z})$ and not just the bounding generators. As the bounding cycles are linear combinations of said basis they will usually automatically be shorter as well. Once the bounding generators are constructed a second optimization step can be applied. See table 3.3 below for the results:

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	15, 5	22, 26, 5, 8	34, 30, 23, 23, 6, 5	23, 8, 14, 21, 34, 6	6, 15, 22, 19	51, 6, 17, 20, 6, 34, 6, 5, 46, 51, 5, 34, 30, 5	28, 26, 45, 5, 18, 5	6, 6, 29, 43, 26, 8, 22, 7, 37, 32
total length:	20	61	121	106	62	316	127	216

Table 3.1: Optimized edge lengths of the cycles in table 2.1

Mesh:	1	2	3	4	5	6	7	8
length of cycles:	5, 15	19, 7, 15, 5	6, 6, 12, 12, 6, 12	5, 16, 14, 14, 5, 5	6, 12, 11, 6	12, 6, 16, 22, 6, 6, 6, 12, 25, 21, 12, 18, 5, 13	6, 16, 8, 5, 15, 15	15, 23, 15, 15, 5, 6, 6, 6, 6, 20
total length:	20	46	54	59	35	180	65	117

Table 3.2: Optimized Edge lengths of the cycles in table 2.4

Mesh:	1	2	3	4	5	6	7	8	9
length of cycles:	15	15, 15,	12, 12, 12,	16, 14 14	12, 11	12, 12, 25, 21, 12, 18, 13	16, 15 15	15, 23, 15, 15, 20	18, 19
total length:	15	30	36	44	23	113	46	88	37

Table 3.3: Edge lengths of the final bounding cycles obtained when using the graph construction strategy from section 2.4 and applying optimization

Chapter 4

Summary and Conclusion

We implemented the algorithm described in Hiptmair and Ostrowski (2002) to construct generators of $H_1(\Gamma_h, \mathbb{Z})$ that are bounding with respect to the exterior. In doing so we also provided visualization of the final generators as well as the different steps of the algorithm. Furthermore we explored multiple spanning tree construction strategies with the aim of creating shorter generators. Lastly we provided a post-processing technique that can further optimize the length of the generators. Consequentially our code goes through the following steps:

i.) Create graph G and dual graph D of the triangulated surface mesh Γ_h

ii.) Create the spanning forest $(T_{D_i})_{i=1}^p$ of D using Kruskal's algorithm with the weights described in section 2.4

iii.) Use BFS from graph centers to create the spanning trees $(T_{G_i'})_{i=1}^p$ of $(G_i')_{i=1}^p$

iv.) Construct the circuits $(s_i)_{i=1,..,N}$ representing a basis of $H_1(\Gamma_h, \mathbb{Z})$ as described in algorithm 1

v.) Shorten the circuits $(s_i)_{i=1,..,N}$ using algorithm 7

vi.) Create polygons $(U_i)_{i=1,..,N}$ representing submerged cycles corresponding to the basis $(s_i)_{i=1,..,N}$

vii.) Use $(s_i)_{i=1,..,N}$ and $(U_i)_{i=1,..,N}$ to calculate the liking numbers and store them in a matrix K

viii.) Compute an integral basis $\mathcal{B}$ of $\mathrm{Ker}(K^\top)$

ix.) Construct generators $(s_i')_{i=1,..,N/2}$ bounding with respect to the exterior as linear combinations of $(s_i)_{i=1,..,N}$ according to $\mathcal{B}$

x.) Shorten the generators $(s_i')_{i=1,..,N/2}$ using algorithm 7

Remark. The method described above is the recommended strategy for spanning tree construction. The code does provide implementations for the other techniques discussed in this paper as well, but beware that using them (in particular DFS) can lead to longer computation time during optimization. Additionally the final bounding cycles obtained after DFS may be hard to comprehend visually, as they frequently visit the same edges multiple times even after optimization.

Bibliography

Colin de Verdière, É. and J. Erickson (2010). Tightening non-simple paths and cycles on surfaces. *SIAM Journal on Computing 39*(8), 3784–3813.

Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein (2009). *Introduction to Algorithms, Third Edition* (3rd ed.). The MIT Press.

Diestel, R. (2017). *Graph Theory* (5th ed.), Volume 173 of *Graduate Texts in Mathematics*. Berlin, Germany: Springer Nature.

Hiptmair, R. and J. Ostrowski (2002, 08). Generators of $H_1(\Gamma_h, \mathbb{Z})$ for triangulated surfaces: Construction and classification. *SIAM Journal on Computing 31*, 1405–1423.

ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz[1] verwendet.

☒ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

Construction, Classification and Optimization of Generators of $H_1(\Gamma_h, \mathbb{Z})$ for Triangulated Surfaces

Verfasst von:
Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):	**Vorname(n):**
Gasparini	Benjamin

Ich bestätige mit meiner Unterschrift:
- Ich habe mich an die Regeln des «Zitierleitfadens» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum	**Unterschrift(en)**
Oetwil am See, 31.5.26	

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

[1] Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html und https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html (Änderungen vorbehalten).